

版权声明，本文档全部内容 & 版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

我是 51CTO 学堂 k8s 和 python 教学总监韩先超，清华大学出版社出版的【Kubernetes 从入门到 DevOps 企业应用实战】畅销书作者，大家百度搜索韩先超，可以看到我详细的介绍，我录制的课程定位是既能学到技术本身，又能用于生产环境，课程内容都是结合公司实际项目讲的，我接触的项目很多，也去过国企，大公司做过架构设计（电信、移动、上海咪咕视讯、天翼物联、深圳任子行等），也对很多公司生产故障进行过现场排查解决，所以这些年积累的经验和实战项目，都会放到我课程里，希望大家跟我一起学习，少走弯路，多挣钱，不断提升自己~

备注：韩先超老师社交平台，建议都关注下，会定期更新不同方向的技术文章和视频：

一、韩先超老师个人微信：luckylucky421302

也可以扫描下方二维码添加：



二、韩先超老师微信公众号：定期发布技术类文章（运维、开发、测试、DBA、网工、渗透安全、K8S 等均覆盖）



三、扫面下面二维码，关注韩老师视频号（定期发布免费技术视频，随时观看）：



四、韩老师抖音号：打开抖音搜索 hxck8s1，定期开直播，发布免费视频，免费分享技术：如 Linux、Docker、K8S、DBA、安全渗透、网路等，扫描下方二维码也可以关注：

版权声明，本文档全部内容&版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。



五、韩老师 b 站账号: bili_55093353115



六、韩先超老师 k8s+devops 书籍购买方法，京东搜索韩先超，可以看到对应书籍

版权声明，本文档全部内容 & 版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**



七、韩老师云原生方向系列课[录播，随时学]->

1、k8s+DevOps 云原生全栈技术（学习人数：170 万+） -

><https://edu.51cto.com/course/27936.html>

2、镜像站被封？不要慌，教你制作自己的镜像，并给 Docker 和 k8s 用（学习人数：10 万+） -><https://edu.51cto.com/course/36466.html>

3、K8S 常见故障速查指南（学习本课程可考取高级云原生工程师-DTC 数字化人才认证） -><https://edu.51cto.com/course/28045.html>

4、kubesphere 管理 k8s 集群-><https://edu.51cto.com/course/38013.html>

5、基于 jenkins+k8s 构建 DevOps 流水线-><https://edu.51cto.com/course/19240.html>

6、istio 微服务【世界 500 强实战精选课程】 -

><https://edu.51cto.com/course/30260.html>

7、k8s 安装（二进制和 kubeadm） -><https://edu.51cto.com/course/28011.html>

8、Rancher 管理 k8s-><https://edu.51cto.com/course/29440.html>

9、k8s 版本升级-><https://edu.51cto.com/course/32890.html>

10、基于生产环境学习 k8s(适合工作的人看)-

><https://edu.51cto.com/course/33653.html>

11、k8s 包管理工具 Helm-><https://edu.51cto.com/course/29441.html>

12、k8s 部署日志组件 EFK-><https://edu.51cto.com/course/29439.html>

13、基于 HPA、VPA、KPA 实现 k8s 弹性伸缩-

><https://edu.51cto.com/course/29446.html>

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

八、韩老师 AI 大模型方向系列课[录播，随时学]->

1、DeepSeek+AI 大模型开发：人工智能+私有化部署+AI 副业变现全攻略-

><https://edu.51cto.com/course/38291.html>

2、【监控革命】Zabbix+DeepSeek 智能监控：AI 驱动的监控实战-

><https://edu.51cto.com/course/38444.html>

3、【AIOps 智能运维】—取代运维岗，又一个新型岗位崛起，未来 10 年就业新方向-

><https://edu.51cto.com/course/38567.html>

九、韩老师职场进阶、面试、写简历方向系列课[录播，随时学]->

1、【职场加速包】：IT 技术人如何打造优秀简历？解锁面试技巧与精准职业规划-

><https://edu.51cto.com/course/36197.html>

2、职场通关第 2 阶段：揭秘企业实战与工作流程【助力成为企业内部核心人员】 -

><https://edu.51cto.com/course/37340.html>

十、韩老师开发方向系列课[录播，随时学]->

1、前端开发课程（HTML、CSS、JS、Vue、React 等） -

><https://edu.51cto.com/course/36377.html>

2、结合 AI 人工智能 chatgpt 学习 go 全栈开发-

><https://edu.51cto.com/course/33969.html>

3、python 基础入门-><https://edu.51cto.com/course/24611.html>

4、python 高级实战-><https://edu.51cto.com/course/24671.html>

十一、韩老师运维方向系列课[录播，随时学]->

1、linux 架构师课程-><https://edu.51cto.com/course/34620.html>

2、k8s 核心技术精讲（50 万+人学习）-><https://edu.51cto.com/course/33081.html>

3、docker 入门到实战-><https://edu.51cto.com/course/28629.html>

4、k8s 部署 redis-><https://edu.51cto.com/course/32888.html>

5、k8s 部署 mysql-><https://edu.51cto.com/course/32887.html>

6、分布式存储 ceph-><https://edu.51cto.com/course/29939.html>

7、prometheus+alertmanager+grafana 监控-

><https://edu.51cto.com/course/22014.html>

十二、韩老师 K8S CKA、CKS 精品课【直播+录播，也可看回放学习模式多样】->

课程采用直播+录播模式进行全方位讲解，融合大厂真实项目，覆盖韩老师对大企业培训的实战项目经验和架构设计，课程不仅讲解 k8s 等云原生等技术，还覆盖 Go 开发等，定位是既能学到技术，又能考取 CKA、CKS 等相关证书。

在线咨询地址-><https://edu.51cto.com/surl=VSmtCT1>

十三、韩老师 K8S CKAD 认证课->

<https://edu.51cto.com/course/35995.html>

十四、韩老师 Python 全栈运维开发精品课【直播+录播，也可看回放学习模式多样】 -

>

在线咨询地址-><https://edu.51cto.com/px/train/1540.html?source=ss>

（覆盖爬虫、devops、人工智能、AI、web 开发（django、flask 等）、游戏开发、自动化运维、个性化脚本和工具开发、全栈开发、数据分析、桌面应用开发等等，很全，python 方向都覆盖了）

一、tkinter 基本介绍：

tkinter 是 Python 的标准 GUI（图形用户界面）库，它提供了创建窗口、按钮、文本框、菜单等 GUI 组件的功能。tkinter 是一个跨平台的工具，可以在 Windows、macOS 和 Linux 等操作系统上运行，使得开发者可以方便地创建桌面应用程序。

主要特点

1. **易于使用**：tkinter 提供了简单的 API，适合初学者和希望快速开发 GUI 应用程序的开发者。
2. **跨平台**：应用程序可以在不同操作系统上运行，提供一致的用户体验。
3. **丰富的组件**：支持多种 GUI 组件，如按钮、标签、文本框、菜单、框架、画布等，能够满足大多数桌面应用的需求。
4. **支持事件驱动编程**：可以轻松处理用户输入和事件，例如鼠标点击、键盘输入等。
5. **可扩展性**：虽然 tkinter 是一个轻量级的库，但它也允许开发者通过其他 Python 库进行扩展。

基本组件

- **窗口 (Tk)**：应用程序的主窗口。
- **标签 (Label)**：用于显示文本或图像。
- **按钮 (Button)**：可点击的按钮。
- **文本框 (Entry 和 Text)**：用于单行或多行文本输入。
- **框架 (Frame)**：用于组织和布局其他组件。
- **菜单 (Menu)**：用于创建应用程序的菜单栏。

项目名称：Tomcat 管理助手



版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

项目描述：

Tomcat 管理助手是一款基于图形用户界面的轻量化管理工具，旨在帮助用户通过简单的操作界面，控制 Tomcat 服务的启动、停止、重启等基本操作。同时，该工具还集成了日志监控功能，实时跟踪 Tomcat 的运行状态，以使用户快速掌握服务情况。工具主要通过系统与命令的交互，实现对 Tomcat 进程的管理，并支持通过图形界面展示日志内容，帮助运维人员高效管理和诊断 Tomcat 服务。

项目功能：

- 1、启动 Tomcat: 一键启动 Tomcat 服务，并显示启动状态。
- 2、停止 Tomcat: 快速停止 Tomcat 服务，避免手动操作繁琐。
- 3、重启 Tomcat: 提供一键重启服务的功能，确保服务平稳重启。
- 4、查看服务状态: 实时监控 Tomcat 的运行状态，通过抓取进程信息来判断服务是否运行。
- 5、日志监控: 实时查看 Tomcat 的日志文件，方便用户掌握服务运行的详细情况。

操作简便：

无需通过终端操作，图形化界面一键完成

一、Tomcat 基本介绍

1. 什么是 Tomcat?

Apache Tomcat 是一个开源的 Web 服务器和 Servlet 容器，它由 Apache 软件基金会开发和维护。Tomcat 的主要功能是执行 Java Servlet 和 JavaServer Pages (JSP) 技术，**允许开发者创建动态的 Web 应用程序。**

2. Tomcat 的组成部分

Tomcat 主要由以下几个组件构成：

- Servlet: Servlet 是 Java 编写的程序，运行在服务器上，处理用户请求并生成响应。它们可以与数据库交互，生成动态内容。
- JSP: JavaServer Pages 是一种基于 Java 的模板引擎，允许开发者在 HTML 中嵌入 Java 代码，简化动态网页的创建。
- Catalina: 这是 Tomcat 的核心组件，负责处理 HTTP 请求并管理 Servlet 的生命周期。
- Coyote: 这是 Tomcat 的连接器，负责处理与客户端（如浏览器）之间的 HTTP 通信。
- Host: Tomcat 可以托管多个 Web 应用程序，每个应用程序都被称为一个“虚拟主机”，Tomcat 使用 Host 来管理这些应用程序。

3. Tomcat 的功能

Tomcat 提供了以下主要功能：

- 支持 Java EE: Tomcat 实现了 Java EE 的 Servlet 和 JSP 规范，使得**开发者可以使用 Java 编写 Web 应用程序。**
- 高效的处理请求: Tomcat 能够处理大量的并发请求，适合大规模的 Web 应用。
- 可扩展性: Tomcat 允许开发者通过添加自定义的 Servlet 和 JSP 来扩展其功能。

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

- 易于部署：开发者可以简单地将 WAR 文件（Web 应用归档）部署到 Tomcat 中，Tomcat 会自动解压和配置应用。

4. Tomcat 的工作原理

Tomcat 的工作流程大致如下：

1. 接收请求：Tomcat 通过 Coyote 接收来自客户端（如浏览器）的 HTTP 请求。
2. 解析请求：请求被传递给 Catalina，Catalina 根据请求的 URL 确定需要调用的 Servlet。
3. 执行 Servlet：对应的 Servlet 处理请求，可能会访问数据库并生成动态内容。
4. 生成响应：Servlet 生成的内容作为 HTTP 响应返回给客户端。
5. 发送响应：Tomcat 通过 Coyote 将响应发送回客户端，完成请求处理。

5. Tomcat 的优点

- 开源免费：Tomcat 是开源的，可以免费使用。
- 跨平台：Tomcat 可以在多种操作系统上运行，如 Windows、Linux 和 macOS。
- 简单易用：Tomcat 的安装和配置相对简单，适合初学者。
- 社区支持：作为一个流行的开源项目，Tomcat 有着活跃的社区和丰富的文档资源。

6. Tomcat 的应用场景

- 生产、开发和测试环境：开发者可以在本地使用 Tomcat 进行 Java Web 应用的开发和测试。
- 小型到中型 Web 应用：Tomcat 适合用于托管小型到中型的 Web 应用程序。

7. Tomcat 命令

- 启动 Tomcat: `catalina.sh start`
- 停止 Tomcat: `catalina.sh stop`
- 重启 Tomcat: `catalina.sh restart`
- 查看 tomcat 状态: `ps -ef |grep tomcat`

二、安装 tomcat 服务

1. 环境准备

确保你的 linux 系统已安装 Java。Tomcat 需要 Java 运行环境（JRE）才能正常工作。

1.1 检查 Java 安装

打开终端并运行以下命令检查是否已安装 Java：

```
java -version
```

如果未安装 Java，可以使用以下命令安装 OpenJDK：

```
sudo yum install java-11-openjdk-devel -y
```

安装完成后，重新运行 `java -version` 以确认安装。

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

2. 下载 Tomcat

访问 [Apache Tomcat 官网](#) 以获取最新版本的 Tomcat 下载链接。或者你可以使用 wget 命令直接下载。

2.1 下载 Tomcat

以下命令示例下载 Tomcat 10（请根据需要修改版本号）：

```
cd /opt
```

```
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.30/bin/apache-tomcat-10.1.30.tar.gz
```

3. 解压 Tomcat

下载完成后，使用以下命令解压 Tomcat 压缩包：

```
sudo tar -zxvf apache-tomcat-10.1.30.tar.gz
```

3.1 重命名目录

为了简化后续操作，可以将 Tomcat 目录重命名：

```
sudo mv apache-tomcat-10.1.30 tomcat
```

4. 配置环境变量

为了方便使用 Tomcat，建议设置环境变量。

4.1 编辑配置文件

打开/etc/profile 文件：

```
sudo vim /etc/profile
```

在文件末尾添加以下内容：

```
export CATALINA_HOME=/opt/tomcat
```

```
export PATH=$PATH:$CATALINA_HOME/bin
```

4.2 使配置生效

运行以下命令使修改后的配置生效：

```
source /etc/profile
```

5. 启动 Tomcat

进入 Tomcat 的 bin 目录并启动 Tomcat 服务：

```
cd /opt/tomcat/bin
```

```
./catalina.sh start
```

6、验证 tomcat 服务是否启动

```
tail -f /opt/tomcat/logs/catalina.out
```

#看 tomcat 启动日志。

浏览器访问安装 tomcat 服务所在机器的 ip:8080 端口：

版权声明，本文档全部内容 & 版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

<http://192.168.40.80:8080/>

三、tomcat.py 代码如下：

```
import tkinter as tk
from tkinter import scrolledtext, messagebox
import paramiko

# 远程服务器的连接信息
HOST = '192.168.40.80' # 远程服务器的 IP 地址
USERNAME = 'root' # SSH 登录的用户名
PASSWORD = '111111' # SSH 登录的密码

# Tomcat 相关操作命令
TOMCAT_START_CMD = "/opt/tomcat/bin/catalina.sh start" # 启动 Tomcat 的命令
TOMCAT_STOP_CMD = "/opt/tomcat/bin/catalina.sh stop" # 停止 Tomcat 的命令
TOMCAT_STATUS_CMD = "ps -ef | grep tomcat | grep -v grep" # 检查 Tomcat 运行状态的命令

# 连接到远程服务器
def connect_ssh():
    try:
        client = paramiko.SSHClient()
        client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        client.connect(HOST, username=USERNAME, password=PASSWORD)
        return client
    except Exception as e:
        log_text.insert(tk.END, f"连接失败: {str(e)}\n")
        return None

# 启动 Tomcat 服务
def start_tomcat():
    client = connect_ssh()
    if client:
        try:
            stdin, stdout, stderr = client.exec_command(TOMCAT_START_CMD)
            error = stderr.read().decode()
            if error:
                log_text.insert(tk.END, f"启动失败: {error}\n")
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

```
        else:
            log_text.insert(tk.END, "Tomcat 启动成功\n")
        finally:
            client.close()

# 停止 Tomcat 服务
def stop_tomcat():
    client = connect_ssh()
    if client:
        try:
            stdin, stdout, stderr = client.exec_command(TOMCAT_STOP_CMD)
            error = stderr.read().decode()
            if error:
                log_text.insert(tk.END, f"停止失败: {error}\n")
            else:
                log_text.insert(tk.END, "Tomcat 已停止\n")
        finally:
            client.close()

# 检查 Tomcat 服务状态
def check_status():
    client = connect_ssh()
    if client:
        try:
            stdin, stdout, stderr = client.exec_command(TOMCAT_STATUS_CMD)
            output = stdout.read().decode()
            error = stderr.read().decode()
            if error:
                log_text.insert(tk.END, f"状态检查失败: {error}\n")
            else:
                if output.strip(): # 如果有输出，说明 Tomcat 正在运行
                    log_text.insert(tk.END, "Tomcat 正在运行\n")
                else:
                    log_text.insert(tk.END, "Tomcat 未运行\n")
                log_text.insert(tk.END, output + "\n")
        finally:
            client.close()

# 创建 GUI 界面
window = tk.Tk()
window.title('Tomcat 管理助手 - 远程')

# 启动按钮
start_button = tk.Button(window, text="启动 Tomcat", command=start_tomcat, width=20)
```

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

```
start_button.grid(row=0, column=0, padx=10, pady=10)
```

停止按钮

```
stop_button = tk.Button(window, text="停止 Tomcat", command=stop_tomcat, width=20)
```

```
stop_button.grid(row=0, column=1, padx=10, pady=10)
```

查看状态按钮

```
status_button = tk.Button(window, text="查看服务状态", command=check_status, width=20)
```

```
status_button.grid(row=1, column=0, padx=10, pady=10)
```

滚动日志窗口

```
log_text = scrolledtext.ScrolledText(window, width=80, height=20)
```

```
log_text.grid(row=2, column=0, columnspan=2, padx=10, pady=10)
```

退出按钮

```
exit_button = tk.Button(window, text="退出", command=window.quit, width=20)
```

```
exit_button.grid(row=1, column=1, padx=10, pady=10)
```

```
window.mainloop()
```

#运行程序:

使用 PyInstaller 打包成 Windows 可执行文件

安装 PyInstaller:

```
pip3 install pyinstaller
```

在项目根目录下运行以下命令，将 Python 脚本打包成 Windows 可执行文件:

```
pyinstaller --onefile --windowed tomcat.py
```

#代码解释说明:

1. 导入所需的库

```
import tkinter as tk
```

```
from tkinter import scrolledtext, messagebox
```

```
import paramiko
```

- tkinter: Python 的标准库，用于**创建图形用户界面 (GUI)**。
- scrolledtext: tkinter 中的一个小部件，用于**创建带有滚动条的文本区域**。
- messagebox: tkinter 模块，用于显示消息框（弹出消息）。
- paramiko: 一个用于与远程计算机建立 SSH2 连接的库，允许执行命令和通过 SSH 传输文件。

2. 定义连接信息

```
HOST = '192.168.40.80' # 远程服务器的 IP 地址
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追究。**

```
USERNAME = 'root'    # SSH 登录的用户名
PASSWORD = '111111'  # SSH 登录的密码
```

这些变量存储了远程 Tomcat 服务器的 IP 地址、用户名和密码。

3. 定义 Tomcat 命令

```
TOMCAT_START_CMD = "/opt/tomcat/bin/catalina.sh start" # 启动 Tomcat 的命令
TOMCAT_STOP_CMD = "/opt/tomcat/bin/catalina.sh stop"  # 停止 Tomcat 的命令
TOMCAT_STATUS_CMD = "ps -ef | grep tomcat | grep -v grep" # 检查 Tomcat 运行状态的命令
```

- 这些命令用于启动、停止和检查 Tomcat 服务器的状态。

4. 连接到远程服务器

```
def connect_ssh():
    try:
        client = paramiko.SSHClient()
        # 创建一个 SSHClient 实例，允许你连接到 SSH 服务器并执行命令
        client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        # set_missing_host_key_policy 方法用于设置主机密钥策略。AutoAddPolicy 会自动接受并
        # 保存新的主机密钥，这在首次连接时非常有用，但在生产环境中可能需要更谨慎。
        client.connect(HOST, username=USERNAME, password=PASSWORD)
        # connect 方法用于连接到 SSH 服务器，参数包括主机名 (HOST)、用户名 (USERNAME)
        # 和密码 (PASSWORD)。
        return client
    # 如果连接成功，函数返回连接的 SSH 客户端对象，以便后续使用。
    except Exception as e:
        # 如果在 try 块中发生任何异常，将执行此部分。
        log_text.insert(tk.END, f"连接失败: {str(e)}\n")
        # 使用 log_text.insert 方法将错误信息插入到一个文本区域（例如，tkinter 的 ScrolledText
        # 小部件）中，以便用户可以查看连接失败的原因。
        return None
    # 如果连接失败，函数返回 None，表示未成功建立 SSH 连接。
```

5. 启动 Tomcat

```
def start_tomcat():
    client = connect_ssh()
    # 调用 connect_ssh() 函数尝试建立 SSH 连接，并将返回的客户端对象存储在变量 client 中。
    if client:
        # 如果连接成功（即 client 不是 None），则执行以下代码块。
        try:
            stdin, stdout, stderr = client.exec_command(TOMCAT_START_CMD)
        # 使用 exec_command 方法在 SSH 服务器上执行启动 Tomcat 的命令
        # (TOMCAT_START_CMD)。该命令通常是启动 Tomcat 服务器的 Shell 命令。stdin、stdout
        # 和 stderr 分别表示标准输入、标准输出和标准错误输出流。
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

```
error = stderr.read().decode()
if error:
    log_text.insert(tk.END, f"启动失败: {error}\n")
#如果 error 不为空，表示启动过程中出现了错误，将错误信息插入到文本区域（如 tkinter
的 ScrolledText 小部件）中进行记录。
else:
    log_text.insert(tk.END, "Tomcat 启动成功\n")
#如果没有错误信息，说明 Tomcat 启动成功，将成功信息插入文本区域。
finally:
    client.close()
```

6. 停止 Tomcat

```
def stop_tomcat():
    client = connect_ssh()
    if client:
        try:
            stdin, stdout, stderr = client.exec_command(TOMCAT_STOP_CMD)
            error = stderr.read().decode()
            if error:
                log_text.insert(tk.END, f"停止失败: {error}\n")
            else:
                log_text.insert(tk.END, "Tomcat 已停止\n")
        finally:
            client.close()
```

- stop_tomcat(): 类似于前一个函数，但用于停止 Tomcat 服务器，并记录结果。

7. 检查 Tomcat 状态

```
def check_status():
    client = connect_ssh()
    #调用 connect_ssh() 函数建立 SSH 连接，并将返回的客户端对象赋值给 client 变量。如
果连接成功，client 将是一个有效的 SSH 客户端对象。
    if client:
        #检查 client 是否有效，只有在成功连接时才会继续执行后面的代码。
        try:
            stdin, stdout, stderr = client.exec_command(TOMCAT_STATUS_CMD)
            #使用 client 对象执行一个命令 TOMCAT_STATUS_CMD，通常这个命令用于检查 Tomcat
            的状态。exec_command 方法返回标准输入、标准输出和标准错误的文件对象。
            output = stdout.read().decode()
            #从标准输出读取命令的结果，并将其解码为字符串，赋值给 output 变量。
            error = stderr.read().decode()
            #从标准错误读取命令执行中的错误信息，并解码为字符串，赋值给 error 变量。
            if error:
```


版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

```
log_text.insert(tk.END, f"状态检查失败: {error}\n")
else:
    if output.strip(): # 如果有输出，说明 Tomcat 正在运行
        log_text.insert(tk.END, "Tomcat 正在运行\n")
    else:
        log_text.insert(tk.END, "Tomcat 未运行\n")
    log_text.insert(tk.END, output + "\n")
finally:
    client.close()
```

8. 创建 GUI

```
window = tk.Tk()
```

#这行代码创建一个 Tk 类的实例，名为 window。Tk 是 tkinter 模块中的一个类，代表一个窗口或应用程序的主窗口。通过创建这个实例，程序将显示一个新的窗口。

```
window.title('Tomcat 管理助手 - 远程')
```

#这行代码设置窗口的标题为 "Tomcat 管理助手 - 远程"。窗口的标题会显示在窗口的标题栏中，通常用于告诉用户这个应用程序的名称或其功能。

9. 向 GUI 添加按钮

启动按钮

```
start_button = tk.Button(window, text="启动 Tomcat", command=start_tomcat, width=20)
start_button.grid(row=0, column=0, padx=10, pady=10)
```

width=20 表示按钮的宽度为 20 个字符单位。

```
start_button.grid(row=0, column=0, padx=10, pady=10)
```

- 这行代码使用 grid 布局管理器将按钮放置在窗口中。
- **row=0**: 指定按钮位于网格的第 0 行。
- **column=0**: 指定按钮位于网格的第 0 列。
- **padx=10**: 为按钮的左右两侧添加 10 像素的内边距。
- **pady=10**: 为按钮的上下两侧添加 10 像素的内边距。

停止按钮

```
stop_button = tk.Button(window, text="停止 Tomcat", command=stop_tomcat, width=20)
stop_button.grid(row=0, column=1, padx=10, pady=10)
```

查看状态按钮

```
status_button = tk.Button(window, text="查看服务状态", command=check_status, width=20)
status_button.grid(row=1, column=0, padx=10, pady=10)
```

滚动日志窗口

```
log_text = scrolledtext.ScrolledText(window, width=80, height=20)
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

- 这一行创建了一个名为 log_text 的可滚动文本区域。
- **scrolledtext.ScrolledText**: ScrolledText 是 tkinter.scrolledtext 模块中的一个类，它扩展了 Text 控件，**添加了滚动条功能**，方便用户查看长文本内容。
- **window**: 将文本区域添加到之前创建的主窗口中。
- **width=80**: 设置文本区域的宽度为 80 个字符单位。
- **height=20**: 设置文本区域的高度为 20 行文本。

```
log_text.grid(row=2, column=0, columnspan=2, padx=10, pady=10)
```

#columnspan=2: 这表示该控件（在这里是 log_text 文本区域）将占用从指定的列（column=0）开始的两个连续列。这意味着它不仅会占用第 0 列，还会占用第 1 列。通过让文本区域跨越两列，控件的宽度会变得更大，这样可以在文本区域内部显示更多的内容，减少用户需要滚动的频率。

退出按钮

```
exit_button = tk.Button(window, text="退出", command=window.quit, width=20)
```

```
exit_button.grid(row=1, column=1, padx=10, pady=10)
```

按钮:

- 创建三个按钮用于启动、停止和查看 Tomcat 状态，每个按钮与相应的功能链接。
- 添加一个 ScrolledText 小部件用于显示日志和输出。
- 包含一个退出按钮用于关闭应用程序。

10. 运行应用程序

```
window.mainloop()
```

- 启动 tkinter 事件循环，等待用户交互，直到窗口关闭。